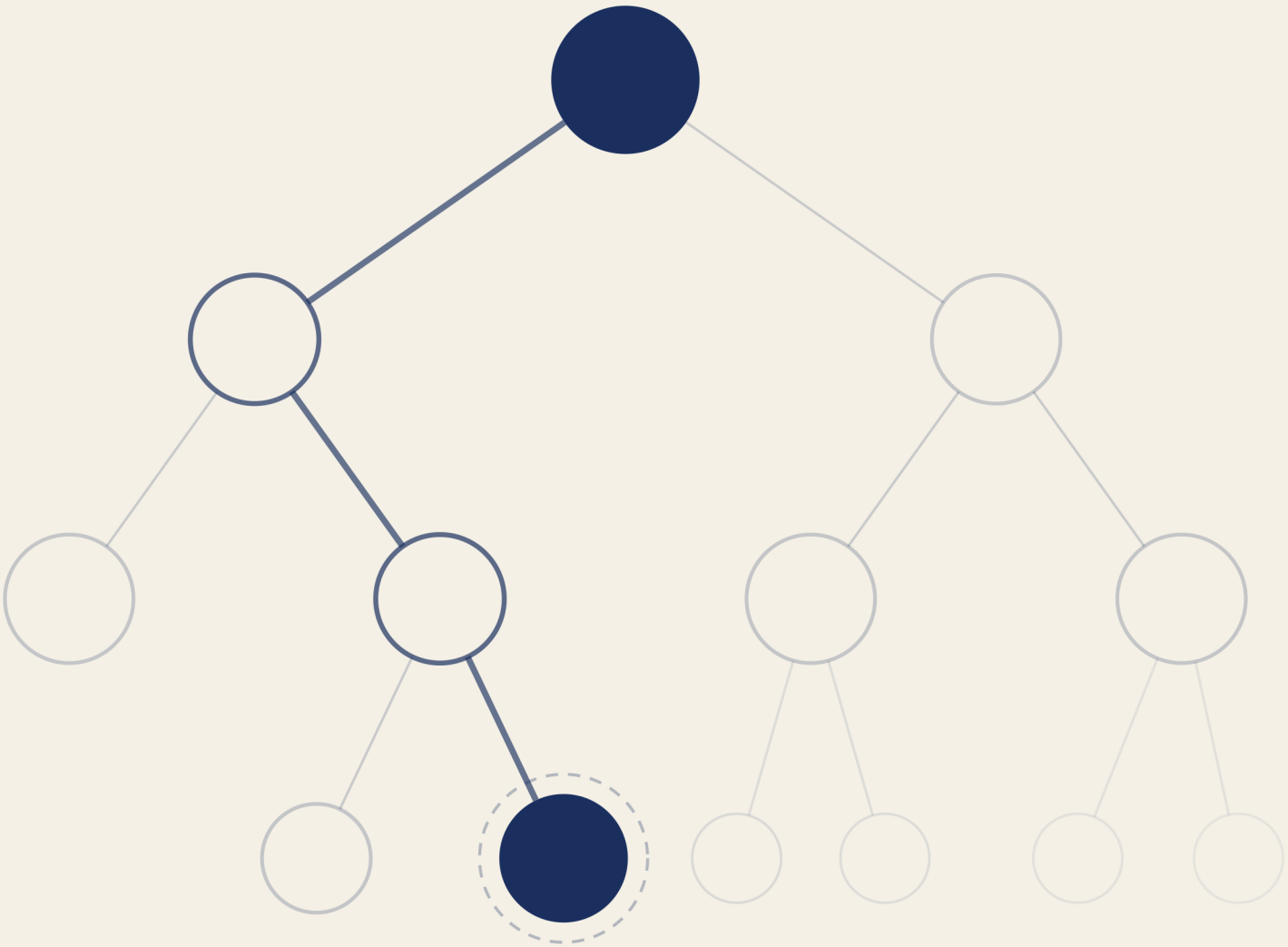
Practical Problem Solving

From Computational Thinking to Programs with Python



Igor Benav

Practical Problem Solving

From Computational Thinking to Programs with Python

Igor Benav

Contents

Preface

Most programming books teach you a language. They walk through syntax, show you features, and hope problem-solving follows. It usually doesn't. You finish the book knowing what a `for` loop is but not when to use one. You can read code but can't design a solution from scratch.

This book reverses the order. We start with problems and introduce Python as the tool to solve them. Every concept exists because a problem demands it, not because it's the next item in a language tour. The goal isn't to memorize syntax. It's to think clearly enough about a problem that the code follows naturally.

This is more important now than it was some years ago. AI tools like ChatGPT can generate working code from a prompt, which means the bottleneck in programming has shifted. Syntax is no longer the hard part. Knowing what to ask for, evaluating whether the result is correct, and understanding why an approach fails at scale: that's the hard part. This book builds those skills. Using AI to help write code is perfectly fine, but you should be able to judge whether a solution is good or broken, and know what to do about it.

About This Book

The book is opinionated. We use Python throughout, uv for package management, and the REPL for quick experiments. We're not surveying every language feature or teaching you the "best" way. We're teaching you one coherent path from "I've never programmed" to "I can solve real problems with code," so you have solid ground to stand on when you explore other directions later.

Computer science concepts (algorithms, abstraction, decomposition) aren't treated as background reading. They're introduced when the problem at hand requires them. You'll understand binary search not because the chapter is titled "Algorithms" but because you're trying to find a name in a phone book and the obvious approach is too slow.

What You'll Build

The book follows a single thread: a receipt printer that starts simple and grows more capable as you learn new concepts. It's not a toy example. It's a program that hits a real wall at the end of every chapter, and the next chapter's concepts are what you need to break through.

In Chapter 2, you build the first version: collecting input, storing it in variables, and printing formatted text. It works, but it can't calculate a total because `input()` returns strings and strings can't do math.

That wall motivates Chapter 3, where you learn about types and conversion. Now the receipt computes totals, applies tax, and formats currency.

But it handles exactly three items, hardcoded. That motivates Chapter 4 (conditionals: skip items with zero quantity, apply discounts based on the subtotal) and Chapter 5 (loops: handle any number of items without duplicating code). Each chapter solves the previous chapter's limitation and reveals the next one.

By the end of the book, you'll have built programs that take input, make decisions, repeat actions, work with structured data, and solve problems you couldn't have approached at the start. More importantly, you'll have a way of thinking about problems that works whether you're writing Python, learning another language, or telling an AI what to build.

How to Use This Book

The chapters are meant to be read in order. Each one builds on the previous. If you skip types, the chapter on conditionals won't make sense. If you skip conditionals, you'll be lost when loops arrive.

Each chapter opens with a problem you can't solve yet. By the end, you can. The concepts exist because the problem demands them: we understand the motivation, see the concept expressed in Python, and then apply it in a hands-on project that extends the running receipt printer (or a related program). This structure was deliberate. It means you always know *why* you're learning something before you learn *how*.

Type the code yourself. Don't copy and paste. The errors you make and fix along the way are part of learning. Change things. Break things. See what happens.

When you get stuck, resist asking AI for the solution. Ask for a hint if you need to, but do the thinking yourself. Struggling is normal. It's also how you actually learn, rather than feeling like you're learning.

One recommendation: don't use an AI-assisted editor like Cursor or Copilot yet. They'll write code for you before you understand what you're writing. Use a plain editor until you can evaluate what the AI produces.

Prerequisites

This book assumes no prior programming experience. You'll need:

- A computer running Windows, macOS, or Linux
- Python 3.9 or newer installed (instructions are provided in Chapter 2)
- A code editor (VS Code works well)
- Willingness to experiment and read error messages instead of panicking

Even if you've never written a line of code before, this book will get you there. Chapter 1 starts with what computer science actually is. Chapter 2 gets Python running.

1 The Foundations of Computational Thinking

People assume computer science is about computers the way they assume astronomy is about telescopes. The telescope is the tool. The stars are the subject. In computer science, the computer is the tool. The subject is problems: how to define them precisely, how to solve them efficiently, and occasionally how to prove they can't be solved at all.

This distinction matters because it shapes everything that follows. You could spend a career writing code without doing much computer science. You could do important computer science with almost no code. What defines the field isn't the medium. It's the questions: What exactly is this problem? Is there an algorithm to solve it? Is there a better one?

Let's start with a problem.

1.1 Finding a Name in a Phone Book

You need to find "Guido" (the creator of Python) in a programmer's phone book with thousands of names. You don't have a search box. How do you do it?

Phone books aren't really used anymore, but the problem maps cleanly to any situation where you're searching a list without built-in search: finding a contact in an old phone, looking up a word in a paper dictionary, scanning a sorted spreadsheet.

One approach: start at the first page. Check the first name. Is it "Guido"? No. Check the second. No. Keep going until you find it or run out of names.

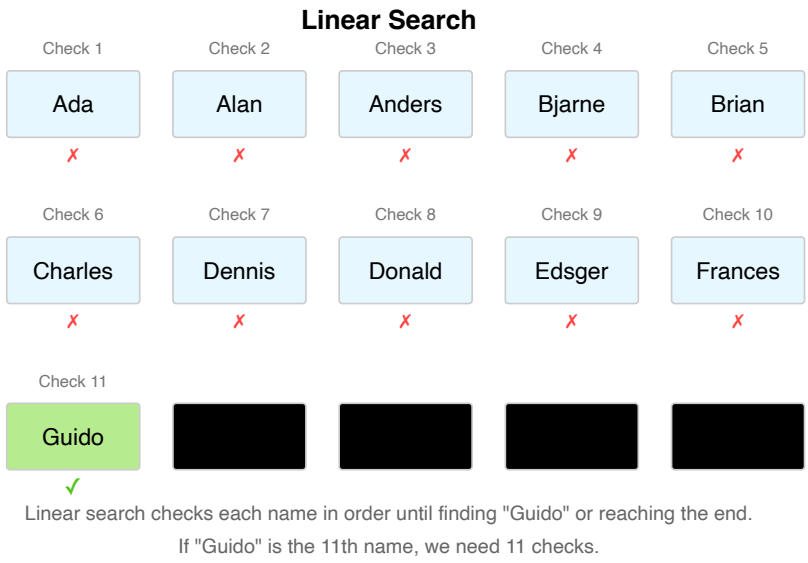


Figure 1.1: Linear Search Visualization

This works. It will always find the name if it's there. But notice what happens as the phone book grows. A phone book with 1,000 names might require 1,000 checks. A million names: a million checks. If checking one name takes one second, a million-entry city phone book could take nearly two weeks. That's not a practical solution. It's a slow one dressed up as a method.

Now think about how you actually use a phone book. You don't start at page one. You flip to the middle. You use the fact that the book is alphabetically ordered. Here's a different approach:

1. Open to the middle of the phone book
2. Check the name at that position
3. If it's "Guido," you're done
4. If "Guido" comes before that name alphabetically, search only the first half
5. If "Guido" comes after, search only the second half
6. Repeat with the chosen half, again splitting it in the middle
7. Continue until you find the name or determine it's not there

Let's trace this on a 20-name phone book. Open to the middle and you land on "James."

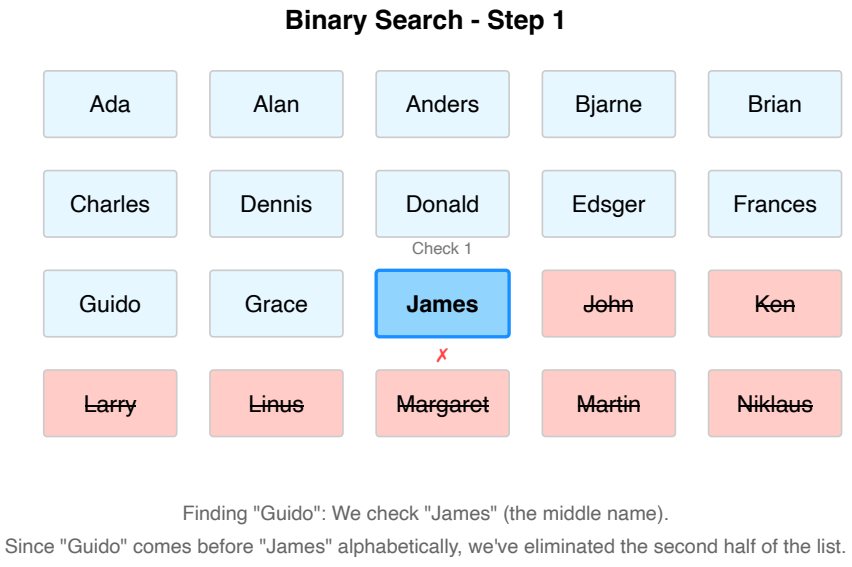


Figure 1.2: Binary Search Step 1

"Guido" comes before "James" alphabetically. Discard the second half. Ten names left. Open to the middle of those: "Donald."

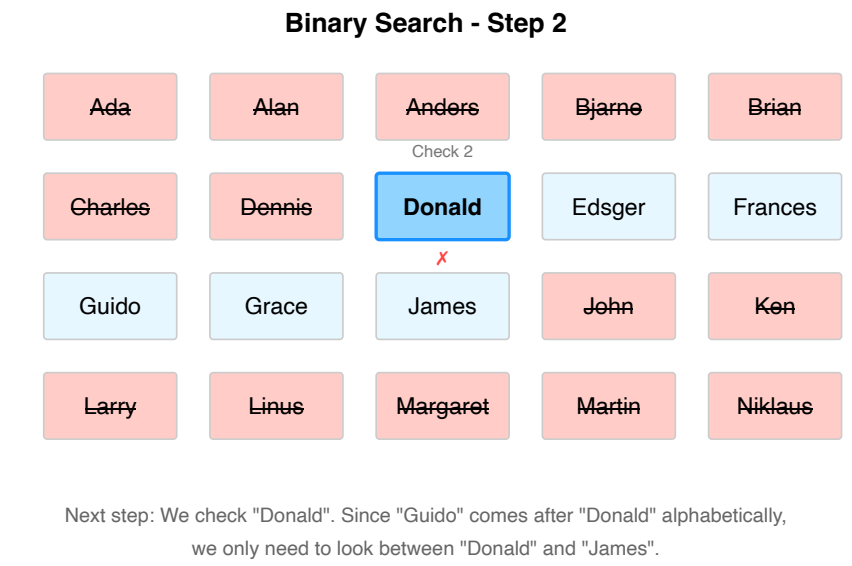


Figure 1.3: Binary Search Step 2

"Guido" comes after "Donald." Down to 5 names, all between "Donald" and "James." Middle of that section: "Frances."

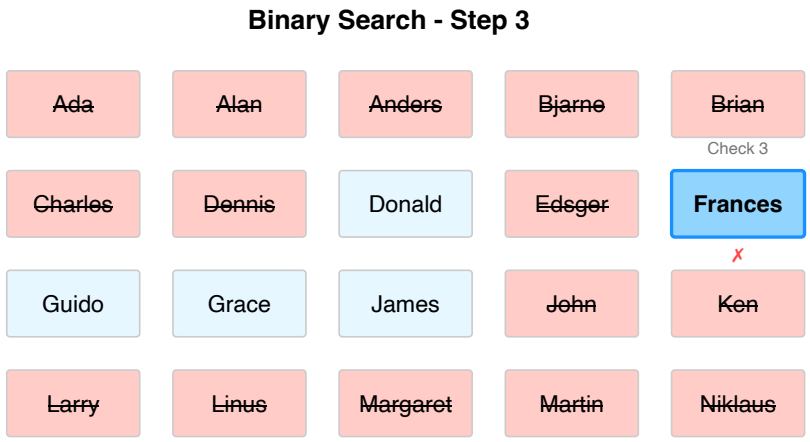


Figure 1.4: Binary Search Step 3

"Guido" comes after "Frances." Two or three names left. Check the middle: "Guido." Found it.

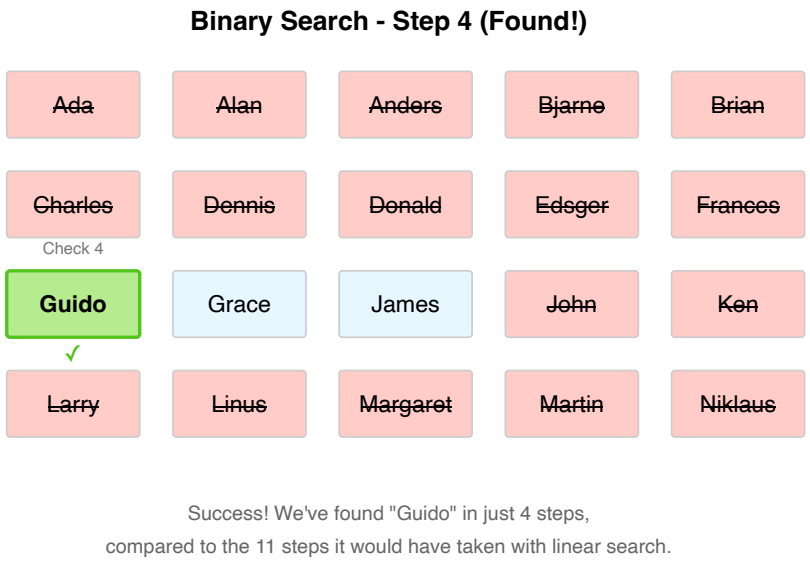


Figure 1.5: Binary Search Step 4

Four checks. The search space halved at every step: 20 names, then 10, then 5, then 2–3, then done. Let's compare the two approaches across phone books of different sizes:

Phone Book Size	Linear Search (worst case)	Binary Search (worst case)
1,000 names	1,000 checks	10 checks
1,000,000 names	1,000,000 checks	20 checks
8,000,000 names	8,000,000 checks	23 checks

For the 8 million name phone book, if each check takes 1 second:
- Linear search: 8 million seconds (over 3 months!)
- Binary search: 23 seconds

Figure 1.6: Search Algorithms Comparison

For 8 million names, if each check takes one second: the first approach takes at most 8 million seconds (over three months). The second takes at most 23 seconds. Both approaches are correct. The difference is whether one is usable.

> Each time you double the phone book's size, the first approach takes twice as long. The second only needs one more step. This is what computer scientists call *logarithmic scaling*, one of the most powerful ideas in algorithm design.

We've been talking about "approaches" and "methods." Computer science has a precise word for what we've described: an **algorithm**. And the difference between the two algorithms we've seen reveals something that matters for everything that follows.

1.2 What Makes an Algorithm?

Both phone book approaches share certain properties. They're sequences of steps. They take an input (a name to find, a phone book to search). They produce an output (the name's location, or the fact that it's not there). They terminate: you always finish, whether or not the name exists.

That's what an **algorithm** is: a finite, well-defined sequence of computational steps that takes an input and produces an output. The word "finite" is doing real work. An algorithm must terminate. A sequence of steps that runs forever isn't a solution; it's a broken process. "Well-defined" matters too. Each step must be unambiguous: precise enough that a machine (or a person following instructions mechanically) could execute it without guessing.

The phone book showed us two algorithms for the same problem. The first is called **linear search**: check each element in sequence. The second is called **binary search**: use the sorted structure to halve the search space at every step. Both are correct. One is dramatically better. That gap matters: for small inputs, efficiency barely matters, but for large inputs, algorithm choice determines whether a solution is practical at all.

Binary search also reveals something subtler. It works because the phone book is sorted. If names were in random order, you couldn't discard half the book with each comparison. The structure of the data enabled a shortcut. This pattern shows up everywhere: finding structure in your problem often unlocks better algorithms.

But there's a gap between understanding an algorithm and being able to explain it precisely. You followed binary search intuitively. Could you write it down well enough for someone who has never seen a phone book?

1.3 The Precision Gap

Consider an algorithm for making a sandwich:

1. Take two slices of bread
2. Spread mayonnaise on one slice
3. Add lettuce, tomato, and cheese
4. Place the second slice on top
5. Cut the sandwich in half

A human understands this instantly. But read it again as if you've never seen a sandwich. Which side of the bread gets the mayonnaise? How much? How should the tomato be sliced? Where does each ingredient go? What does "cut in half" mean: diagonally? Down the middle?

You fill in these gaps without thinking. You've made sandwiches, you know what bread is, you have common sense. A computer has none of that. It needs exact quantities, exact positions, exact actions. Every unstated assumption, every common-sense inference, has to be made explicit.

Go back to binary search. "Open to the middle of the phone book" is clear enough for you. But a computer needs to know: how do you compute "the middle" of a list? If the list has an even number of entries, which of the two middle entries do you check? When you "discard the second half," do you include the entry you checked, or not? These details don't change whether the algorithm works in principle, but they're the difference between an idea and an implementation.

This is what makes programming difficult when you're starting out. It's not that the problems are hard. It's that you're shifting from implicit to explicit: you have to say what you mean completely, not rely on the reader to fill in the rest. A machine that follows instructions exactly (with no assumptions, no improvisation) can do it billions of times without fatigue. But it will only do exactly what you said, not what you meant.

The phone book gave us two algorithms and showed that algorithm choice matters. The sandwich showed that precision matters. But what happens when the problem is bigger than a single search?

1.4 Breaking Problems Apart

Finding a name in a phone book is one task. Building a contacts app involves many: adding new contacts, sorting them, searching by name, searching by number, handling duplicates, displaying results. Each of these is its own problem with its own algorithm. The skill is recognizing where a large problem divides into smaller, independent pieces.

This is **decomposition**: breaking a complex problem into subproblems, each manageable on its own. It mirrors what binary search does to the phone book (split the problem in half, solve the smaller version), but applied to the problem's structure rather than its data. A hard problem is often several easier problems stacked together.

Decomposition alone isn't enough, though. Once you've broken a problem into pieces, you need a way to work on each piece without drowning in the details of all the others.

1.5 Abstraction

When you described binary search to a friend, you probably said something like "open to the middle, check the name, throw away half." You didn't explain how pages work, how your eyes scan text, or how alphabetical ordering is defined. You worked at a level where those details didn't matter.

This is **abstraction**: hiding what you don't need to think about so you can focus on what you do.

When you use a smartphone, you don't think about transistors or voltage levels or memory addresses. You tap an icon. The icon is an abstraction: an interface that hides enormous complexity underneath. You don't need to understand what's happening at the hardware level to use the app. The abstraction lets you work at a higher level.

Programming languages work the same way. When you call `print()` in Python, you're not writing the machine instructions that move characters to a screen buffer. You're using an abstraction that handles all of that for you. Python itself is an abstraction over assembly code, which is an abstraction over machine code, which is an abstraction over electrical signals running through the processor.

Binary search has layers too. At the top: "halve the search space until you find the target." One level down: "compare the target to the middle element and decide which half to keep." One level further: "compute the index of the middle element given the current bounds." You can think about the strategy without thinking about index arithmetic, and you can think about index arithmetic without thinking about how memory stores the list. Each layer trusts the one below it.

This layering is how complex systems get built at all. You can work at one level without understanding the levels below. When you need to go deeper, you can. Most of the time, the abstraction holds and you don't have to.

We've been describing algorithms in words and diagrams. At some point they need to run on a computer. That requires translating them into a language a computer can execute.

1.6 From Algorithms to Programs

An **algorithm** is a logical sequence of steps to solve a problem: language-independent, the idea itself. A **program** is that algorithm implemented in a specific programming language: one particular expression of the idea.

Binary search is an algorithm. You could implement it in Python, Java, C++, or any other language. The algorithm stays the same; the syntax changes. Programming is the act of transcribing an algorithm into a language a computer can execute. Without a clear algorithm, you're not writing a program; you're improvising in code, which produces software that works by accident and fails in ways you don't understand.

This wasn't always done in high-level languages. Early programmers gave instructions directly in machine code: the ones and zeros the processor actually understands. A program to display "Hello, World!" in machine code looks like this:

```
B4 03 CD 10 B0 01 B3 0A B9 0D 00 BD 13 01 B4 13 CD
10 C3 48 65 6C 6C 6F 2C 20 57 6F 72 6C 64 21 0D 0A
```

Broken down, those hexadecimal codes represent specific processor instructions:

```
B4 03       ; MOV AH, 03h     (Get cursor position)
CD 10       ; INT 10h         (BIOS video interrupt)
B0 01       ; MOV AL, 01h     (Write mode parameter)
B3 0A       ; MOV BL, 0Ah     (Color attribute)
B9 0D 00    ; MOV CX, 000Dh   (Character count - 13 for "Hello, World!")
BD 13 01    ; MOV BP, 0113h   (Offset to string data)
B4 13       ; MOV AH, 13h     (Write string function)
CD 10       ; INT 10h         (BIOS video interrupt)
C3          ; RET             (Return)
```

And the text data itself, encoded as numbers:

```
48 65 6C 6C 6F 2C 20 57 6F 72 6C 64 21 0D 0A
```

You don't need to understand this. You need to feel the gap between that and the Python equivalent:

```python
print("Hello, World!")
```

Same result. One line versus dozens of cryptic instructions. That gap is what high-level programming languages exist to close (and it's abstraction at work: `print()` hides all of that machine-level complexity behind a single word).

Here's another example. This algorithm finds the maximum value in a list:

1. Start with the first number as the current maximum
2. For each remaining number, if it's larger than the current maximum, it becomes the new maximum
3. After checking all numbers, the current maximum is the answer

And here's what it looks like in Python (don't worry about the syntax for now):

```
def find_maximum(numbers):
    if not numbers:
        return None

    maximum = numbers[0]

    for number in numbers[1:]:
        if number > maximum:
            maximum = number

    return maximum

my_numbers = [42, 17, 8, 94, 23, 61]
print(f"The maximum value is {find_maximum(my_numbers)}")
```

The algorithm is the logic. The Python code is one way to express it. Keeping these separate in your head matters: when your program gives the wrong answer, the problem is either in the algorithm (the logic is wrong) or in the implementation (the code doesn't correctly express the logic). Those are different bugs with different fixes, and you won't find either one by staring at the wrong thing.

The choice of which language to use is a separate question, and for this book, it's already been made.

1.7 Why Python?

Python was created by Guido van Rossum in the late 1980s. Its central design philosophy, summarized in a document called "The Zen of Python," includes lines like:

- Beautiful is better than ugly
- Explicit is better than implicit
- Simple is better than complex
- Readability counts

These aren't aesthetic preferences. They make Python code read like pseudocode: instructions close enough to plain English that the algorithm is visible in the implementation. For learning, that's valuable. You can focus on the problem without fighting the language.

Python is also practical. It's the dominant language in data science, machine learning, scientific computing, and automation. Learning it doesn't teach you to program; it gives you access to a vast ecosystem of tools built by other people solving related problems.

It has real limitations, and it's worth being honest about them. Python is interpreted: it's translated to machine instructions at runtime rather than compiled in advance. Compiled languages like C++ or

Rust are typically much faster. For most applications this doesn't matter, but for performance-critical systems, it does. Python also has a mechanism called the Global Interpreter Lock (the **GIL**) that allows only one thread to execute Python code at a time, which limits certain kinds of parallel processing. And some domains (mobile apps, game engines, low-level systems programming) are better served by other languages. Every language is a tradeoff. Python is optimized for programmer productivity and the scientific and data ecosystem.

Professional programmers typically know several languages and choose based on the problem. The concepts you'll learn here (algorithms, data structures, abstraction) apply in any language. Python is a clear place to start.

The concepts matter more than the tool. Which raises one more question: what kinds of problems can these concepts actually solve?

1.8 What Makes a Problem Computational?

Not every problem can be solved with an algorithm. We've been working with "find a name in a phone book," which is clearly solvable: there's a finite list, a defined target, and a procedure that terminates. But consider "what is the most beautiful painting?" There's no unambiguous definition of "beautiful," no finite set of candidates everyone agrees on, and no procedure that would produce an answer everyone accepts. It's not a computational problem.

For a problem to be computational, it needs to be well-defined (the problem statement is clear and unambiguous), solvable (there exists a finite sequence of steps that reaches an answer), and finite (the solution must terminate). "What is the sum of all integers from 1 to 100?" is computational. "What career would make someone happiest?" is not.

Computational thinking is the practice of reformulating problems so they become solvable algorithmically. This often involves ignoring irrelevant details (abstraction), breaking the problem into pieces (decomposition), and recognizing when a new problem resembles one you've already solved (pattern recognition). It's the collection of skills we've been building throughout this chapter, applied deliberately.

The phone book showed us all of this in miniature. We started with a problem. We found two algorithms and discovered that the choice between them matters enormously. We saw that the data's structure (sorted order) enabled the better algorithm. We noticed that describing an algorithm precisely enough for a machine requires closing the gap between human intuition and explicit instructions. We used decomposition and abstraction to manage complexity. And we distinguished the algorithm (the idea) from the program (its implementation in a specific language).

These are the ideas the rest of the book applies. In the next chapter, we move from theory to practice: setting up Python and writing actual programs. The thinking here is what makes that code meaningful.

1.9 Chapter Summary

Computer science is primarily about solving problems, not about computers. The phone book made the stakes concrete: linear search and binary search both find the name, but at scale the difference between them is the difference between seconds and months. Algorithm choice dominates.

An algorithm is a finite, well-defined sequence of steps that takes an input and produces an output. "Finite" and "well-defined" are both essential. Binary search also showed that the structure of the data (sorted order) enabled the better algorithm. Finding structure in your problem is often what unlocks a better solution.

The sandwich example exposed the precision gap: humans tolerate vague instructions, computers don't. Programming requires you to close that gap, stating every step explicitly enough that a machine can follow without guessing. Decomposition breaks large problems into manageable pieces. Abstraction lets you work at one level without drowning in the details of the levels below. Both are how complex systems get built at all.

Algorithms are language-independent descriptions of solutions. Programs are their implementation in a specific language. Keeping those two things distinct makes debugging much cleaner: when something goes wrong, is the logic flawed or did the code fail to express the logic correctly?

In the next chapter, we set up Python and write actual programs. The exercises below are worth doing even if they seem obvious. They're not testing what you know. They're wiring the circuits.

1.10 Exercises

1. Write a detailed algorithm for a common daily task: making breakfast, getting ready in the morning, or anything you do regularly. Be precise enough that someone who has never done it could follow your instructions without guessing. Once you're done, go back and ask: where did you make assumptions? What did you leave implicit that a computer would need stated explicitly? The gap between your first draft and a truly unambiguous version is the gap that programming requires you to close.
2. Using the sorted list of 16 programming languages below, pick any language other than Python and trace both algorithms to find it. For linear search, check each language in order and count the comparisons. For binary search, use divide-and-conquer and count the comparisons. Write out every step of both. The difference in step counts is the point, but try to also articulate *why* binary search needs so many fewer steps.

```
1. Ada        5. Cobol    9. JavaScript  13. Python
2. C          6. Fortran  10. Kotlin      14. Ruby
3. C#         7. Go       11. Perl        15. Rust
4. C++        8. Java     12. PHP         16. Swift
```

3. Choose one of these complex tasks and break it into subtasks, each precise enough to be its own algorithm: planning a vacation, organizing a birthday party, or writing a research paper. The goal isn't exhaustiveness; it's identifying the natural seams in the problem, the places where a complex task divides into independent pieces. How deep do you need to go before each piece feels manageable?

4. For each of the following, decide whether it's computational (solvable by algorithm) or not, and explain your reasoning: (a) finding the shortest route between two cities, (b) determining the most beautiful musical composition, (c) calculating the average temperature for a city over a year, (d) converting currencies based on current exchange rates, (e) deciding which career would make someone happiest. Some of these are less obvious than they first appear.

5. Below is a 28-step grocery shopping trip written out in full. Group the steps into meaningful functions and rewrite the whole trip using only those function names, so the high-level description fits in a few lines. The complexity lives inside the named abstractions, not at the top level. Then ask yourself: does the abstraction actually hide the right things? Are there groupings that feel arbitrary, and why?

```
1. Write down needed items
2. Check what you already have at home
3. Update shopping list
4. Get reusable bags
5. Find car keys
6. Drive to grocery store
7. Park the car
8. Grab a shopping cart
9. Enter the store
10. Check first item on list
11. Find the correct aisle
12. Locate the item
13. Check price and quality
14. Place item in cart
15. Cross item off list
16. Repeat steps 10-15 until list is complete
17. Proceed to checkout
18. Wait in line
19. Place items on conveyor belt
20. Pay for groceries
21. Bag the groceries
22. Return cart
23. Walk to car
24. Load bags into car
25. Drive home
26. Carry bags inside
27. Unpack groceries
28. Store items in proper places
```

6. The binary search algorithm in this chapter relies on the phone book being sorted. What would happen if it weren't? Try tracing binary search on an unsorted list of 8 names (make one up). At what point does the algorithm break down? What does this tell you about the relationship between an algorithm and the assumptions it makes about its input?

2 Getting Started with Python

Chapter 1 ended with the gap between an algorithm and a program. You understand binary search. You could explain it to someone. But explaining it to a computer requires a level of precision that English doesn't demand: exact syntax, explicit instructions, no unstated assumptions. Closing that gap is what this chapter is about.

We'll set up Python, learn how to run code, and build a small program that takes input from a user, processes it, and produces formatted output. That input-process-output loop is the structure of most programs you'll ever write. By the end of the chapter you'll have written one, and you'll have hit a limitation that motivates everything in Chapter 3.

2.1 Installing Python

Python runs on Windows, macOS, and Linux. The installation process is slightly different on each, but the result is the same: a `python` command you can run from your terminal.

On Windows, go to python.org/downloads and download the installer for the latest version (3.9 or later works for this book). Run it, and on the first screen check the box that says "Add Python to PATH" before clicking anything else. Without that, Windows won't know where to find Python when you type its name. After installation, open Command Prompt and run `python --version`. If you see a version number, you're done.

On macOS, the system ships with Python, but it's usually outdated and managed by the OS in ways that can cause trouble. Install a fresh copy with Homebrew: run `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"` in Terminal to get Homebrew, then `brew install python`. Verify with `python3 --version`.

On Linux, Python is almost certainly already installed. Check with `python3 --version`. If it's missing or very old, use your package manager: `sudo apt install python3` on Debian/Ubuntu, `sudo dnf install python3` on Fedora, `sudo pacman -S python` on Arch.

For a code editor, Visual Studio Code is the right choice for this book. It's free, runs on all platforms, and has solid Python support through its Python extension. Download it from code.visualstudio.com and install the Python extension from the Extensions panel. If you can't install software locally, pydantic.run is an online Python environment that works in a browser with no setup at all. It's a good fallback for getting started, though you'll want a local setup eventually for the later chapters.

One recommendation: don't use an AI-assisted editor like Cursor or Copilot yet. They'll write code for you before you understand what you're writing, which makes the early chapters pointless. Use a plain editor until you can evaluate what the AI produces.

Once Python is installed, there are two ways to run it. Understanding both matters.

2.2 Two Ways to Run Code

The first way is interactive. Open a terminal, type `python` (or `python3` on macOS/Linux), and you'll see a prompt that looks like >>>. This is the **REPL** (Read-Evaluate-Print Loop): you type an expression, Python evaluates it, prints the result, and waits for the next input.

```
>>> 2 + 3
5
>>> "Hello" + " " + "World"
'Hello World'
```

The REPL is useful for testing small ideas quickly: does this expression do what I think? You'll use it throughout the book to check your understanding before committing something to a file. Type `exit()` or press Ctrl+D (Ctrl+Z on Windows) to leave it.

The second way is script mode. You write code in a file with a `.py` extension and run the whole file at once. Create a file called `first.py` with this content:

```
2 + 3
"Hello" + " " + "World"
```

Run it with `python first.py` (or `python3 first.py`). Nothing appears. That's not a bug: in script mode, Python evaluates expressions but doesn't print results unless you explicitly ask it to. This is the first taste of the precision gap from Chapter 1. You assumed Python would show you the result. Python did exactly what you said, which was "compute this value." You never said "show it to me."

That's what `print()` is for.

2.3 Telling the Computer What to Show

Fix the script:

```
print(2 + 3)
print("Hello" + " " + "World")
```

Now it outputs:

```
5
Hello World
```

`print()` takes whatever you give it and displays it on screen. It's how your program communicates. Without it, the program computes in silence.

You can print multiple values in a single call by separating them with commas. Python inserts a space between them:

```
print("The answer is", 42)
```

```
The answer is 42
```

By default `print()` adds a newline after each call. You can change that with the `end` parameter: `print("Hello", end=" ")` leaves the cursor on the same line so the next `print()` continues there.

Now we can display things. But everything so far is hardcoded: the program does the same thing every time. To build anything useful, we need to work with values that change. We need the computer to remember things.

2.4 Remembering Values

A **variable** is a name that refers to a value. You create one with the assignment operator =:

```
message = "Hello, Python!"
number = 42
print(message)
print(number)
```

```
Hello, Python!
42
```

The left side is the name, the right side is the value. Once assigned, the name can be used anywhere you'd use the value directly. This is how programs hold onto information: you compute something, give it a name, and use that name later.

Variables can be reassigned:

```
count = 10
count = count + 1
print(count)
```

```
11
```

The second line reads the current value of `count`, adds 1, and assigns the result back to `count`. This pattern is constant in programming: it's how counters, accumulators, and running totals work.

Variable names can contain letters, numbers, and underscores, but can't start with a number and can't be Python keywords like `if` or `for`. Python convention is lowercase words separated by underscores: `user_name`, `total_score`, `highest_temperature`. Names are case-sensitive: `score` and `Score` are different variables. Use descriptive names (`user_age` instead of `x`). You'll thank yourself later.

Now we can display things and remember them. But the program still does the same thing every time it runs, because every value is written into the code. A useful program responds to whoever is using it. It needs to ask questions.

2.5 Asking the User

`input()` pauses the program, displays a prompt, waits for the user to type something and press Enter, then returns what they typed as a string.

```
name = input("What is your name? ")
print("Hello, " + name + "!")
```

When this runs, the user sees `What is your name?` and can type a response. Whatever they type gets stored in `name` and used in the greeting. This is the first program we've written that behaves differently depending on who runs it. The same code, different input, different output.

One thing to keep in mind: `input()` always returns a string, even if the user types a number. If you ask someone for their age and they type `25`, you get the string `"25"`, not the integer `25`. This distinction between text and numbers will matter very soon.

We now have four tools: `print()` to display output, expressions to compute values, variables to remember them, and `input()` to get information from the user. Before we combine them, we need to understand how Python handles the two main kinds of data we'll work with: numbers and text.

2.6 Numbers and Text

Python supports the arithmetic operators you'd expect: `+` for addition, `-` for subtraction, `*` for multiplication, `/` for division. Division always returns a decimal, even when the result is a whole number: `10 / 2` gives `5.0`, not `5`. Two additional operators are worth knowing early: `**` for exponentiation (`2 ** 8` gives `256`) and `%` for the remainder after division (`10 % 3` gives `1`).

The order of operations follows standard math: exponentiation first, then multiplication and division, then addition and subtraction. Parentheses override the default order: `2 + 3 * 4` gives `14`, but `(2 + 3) * 4` gives `20`.

Text in Python is called a **string**. You write strings in either single or double quotes; the choice doesn't matter as long as you're consistent within a single string. Strings support two operators that look familiar but behave differently. `+` concatenates them: `"Hello" + " " + "World"` gives `"Hello World"`. `*` repeats them: `"ha" * 3` gives `"hahaha"`.

Notice the difference. When Python sees `2 + 3`, it adds the numbers and gets `5`. When it sees `"Hello" + " World"`, it glues the strings together and gets `"Hello World"`. The same operator, different behavior depending on the data type. This is important: `"2" + "3"` gives `"23"`, not `5`. The quotes make them strings, so `+` concatenates instead of adding.

This is where `input()` becomes tricky. Remember: it always returns a string. If the user types `5` and you store it in a variable called `price`, you have the string `"5"`, not the number `5`. You can concatenate it with other strings (`"Price: $" + price` gives `"Price: $5"`), but you can't do math with it. Not yet. We'll fix this in Chapter 3 when we introduce type conversion.

That's the full toolkit for this chapter: output, expressions, variables, input, and the beginnings of understanding data types. Time to use them together.

2.7 Hands-On: Personal Receipt Printer

We'll build a program that collects information about a purchase, formats it, and prints a receipt. It uses every concept from the chapter and produces something you can look at and modify. It also runs straight into a wall that will make the need for Chapter 3 concrete.

Set up your project:

```
mkdir chapter2
cd chapter2
uv init
```

Create a file called `receipt.py`. We'll build it up in pieces.

Start by collecting the input:

```
# receipt.py
print("=== Receipt Printer ===")
print()

store_name = input("Store name: ")
item_name = input("Item name: ")
quantity = input("Quantity: ")
price = input("Price per item: ")
customer_name = input("Customer name: ")
```

Run it with `uv run python receipt.py`. You should be prompted for each piece of information in order. The program collects it but doesn't do anything with it yet. Five variables, five `input()` calls, five values stored and waiting.

Now add the receipt output below the input section:

```
# receipt.py
# ...existing code above

print()
print("=" * 30)
print("           " + store_name)
print("=" * 30)
print()
print("Customer: " + customer_name)
print()
print(item_name)
print("  " + quantity + " x $" + price)
print()
print("-" * 30)
print("  Total items: " + quantity)
print("=" * 30)
print("    Thank you for your purchase!")
print("=" * 30)
```

Run it again and enter some values. If you enter "Bookshop" as the store, "Book" as the item, "2" as the quantity, and "14.99" as the price, the receipt looks like:

```
==============================
           Bookshop
==============================

Customer: Alice

Book
  2 x $14.99

------------------------------
  Total items: 2
==============================
    Thank you for your purchase!
==============================
```

Notice that `"=" * 30` produces a row of thirty equals signs. The `*` operator on strings repeats the string, which is a common way to draw lines in terminal output.

Now try to add a total. You have the quantity and the price. The total should be quantity times price. Try adding this line:

```
print("  Total: $" + quantity * price)
```

Run it. Python crashes. The error says something about not being able to multiply a string by a string. And that's the point. `quantity` and `price` are strings (because `input()` always returns strings),

and Python doesn't know how to multiply `"2"` by `"14.99"`. You can't compute `"2" * "14.99"` any more than you can compute `"hello" * "world"`. The operation doesn't make sense for text.

You might try to work around it:

```
print("  Total: $" + quantity * int(price))
```

That doesn't work either: `int()` can't convert `"14.99"` because it has a decimal point. And even if the price were a whole number, `"2" * 3` repeats the string (`"222"`), which isn't the math you want.

The receipt captures the pattern (collect input, build strings, format output), but it can't do the one thing a real receipt needs: calculate the total. We're treating numbers as text, and text can't do arithmetic. Chapter 3 fixes this by introducing data types properly: what distinguishes integers from floats from strings, how to convert between them, and what operations make sense on each. Once you can write `float(price) * int(quantity)`, the receipt works for real.

Remove the broken total line for now. The receipt is complete as a string-formatting exercise. Try modifying it: change the formatting, add a date line, widen the receipt. Breaking things and seeing what happens is how you actually learn what the rules are.

2.8 Chapter Summary

Setting up Python is overhead, but it only happens once. The two modes of running code (interactive and script) serve different purposes: the REPL for quick experiments, script files for anything you want to keep. The first surprise was that script mode doesn't show results unless you ask: Python does what you say, not what you assume.

`print()` is how programs communicate output; `input()` is how they accept it. Variables store values so they can be reused and combined. These are the minimum tools for the input-process-output loop that most programs follow. The receipt printer demonstrated that loop, but it also demonstrated a wall: `input()` returns strings, and strings can't do math. The program can display prices but can't add them up, can show quantities but can't multiply them.

That wall is the data type system. In the next chapter, we'll introduce types properly: integers, floats, strings, and the conversions between them. That's what turns text into numbers and makes computation possible.

2.9 Exercises

1. Extend the receipt printer to support three line items instead of one. Ask for a name, quantity, and price for each item separately. Format them as three rows in the receipt body. You'll have three `item_name`, `quantity`, and `price` variables (name them clearly: `item1_name`, `item2_name`, etc.). The total line still can't be calculated yet: display the individual quantities. Which part of the output is hardest to format correctly, and why?

2. Write a program that asks for a sender's name, a recipient's name, and a one-sentence message, then formats and prints a postcard. The postcard should have a border made of a repeated character, the recipient prominently at the top, the message in the middle, and "From: [sender]" at the bottom. The border width should be determined by the length of the message (`"=" * len(message)` gives you a line exactly as wide as the message). What happens if the message is very short or very long?

3. Write a program that asks for someone's first name, last name, and preferred title (Mr., Ms., Dr., etc.), then prints their name formatted three ways: full formal (`Dr. Jane Smith`), last name first (`Smith, Jane`), and initials (`J.S.`). The initials require you to access individual characters in a string: `name[0]` gives you the first character. What happens if you use `name[0]` on an empty string?

4. The `print()` function accepts a `sep` parameter that controls what gets placed between multiple values instead of the default space: `print("a", "b", "c", sep="-")` prints `a-b-c`. Write a program that asks for five words and prints them in three formats: space-separated (default), comma-separated, and with a pipe character between them (`word1|word2|word3`). Now try passing all five variables to a single `print()` call with the appropriate `sep`. How does that compare to building the string manually with +?

5. Write a program that generates a name tag for a conference badge. Ask for the person's name, their job title, and the company they work for. The badge should have a fixed width of 30 characters, with each line of text centered. Python strings have a `.center(width)` method: `"hello".center(30)` pads the string with spaces so it's centered in a field of 30 characters. Use this to format the badge. What happens when a name is longer than 30 characters?

6. Write a program that asks for two words and prints every combination of them with a separator between them: the first word, then the second, then both joined, then the second first and the first second, then the first repeated twice, then the second repeated twice. Six lines of output from two inputs. Before writing any code, write out what the six lines should look like for inputs `"black"` and `"bird"`. Does the output match what you expected? The goal isn't to memorize string operations; it's to notice whether your mental model of what + and * do on strings matches what Python actually does.

Downey, Allen B. 2015. *Think Python: How to Think Like a Computer Scientist*. 2nd ed. O'Reilly Media.

Guttag, John. 2016. *Introduction to Computation and Programming Using Python: With Application to Understanding Data*. 2nd ed. MIT Press.

Hammack, Richard H. 2020. *Book of Proof*. 3rd ed. Virginia Commonwealth University.

Percival, Harry J. W., and Bob Gregory. 2020. *Architecture Patterns with Python: Enabling Test-Driven Development, Domain-Driven Design, and Event-Driven Microservices*. O'Reilly Media.

Pólya, George. 2014. *How to Solve It: A New Aspect of Mathematical Method*. 2nd ed. Princeton University Press.

Ramalho, Luciano. 2021. *Fluent Python: Clear, Concise, and Effective Programming*. 2nd ed. O'Reilly Media.

Robson, Patrick. 2021. *Robust Python: Write Clean and Maintainable Code*. O'Reilly Media.

Shaw, Zed A. 2013. *Learn Python the Hard Way: A Very Simple Introduction to the Terrifyingly Beautiful World of Computers and Code*. 3rd ed. Zed Shaw's Hard Way Series. Addison-Wesley Professional.

Slatkin, Brett. 2019. *Effective Python: 125 Specific Ways to Write Better Python*. 2nd ed. Effective Software Development Series. Addison-Wesley Professional.

Sweigart, Al. 2019. *Automate the Boring Stuff with Python: Practical Programming for Total Beginners*. 2nd ed. No Starch Press.

Velleman, Daniel J. 2019. *How to Prove It: A Structured Approach*. 3rd ed. Cambridge University Press.